Fiche patterns – Arbres généraux

Tree : `T.key`, `T.children` (liste), `T.nbchildren`

TreeAsBin : `B.key`, `B.child`, `B.sibling` **Queue** : `enqueue`, `dequeue`, `isempty`

Étape 1 – BFS ou DFS ? Un seul mot décide

La règle

Le mot **niveau** (ou **largeur**) est dans l'énoncé ⇒ **BFS**.

Sinon ⇒ **DFS** (c'est le choix **par défaut**).

Exception : le sujet **impose** le parcours (« en profondeur ») ⇒ on obéit.

Mots qui veulent dire BFS

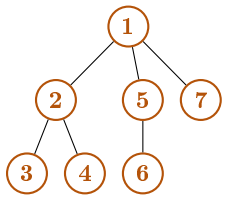
- **niveau**, par niveau, chaque niveau
- **largeur**
- même **profondeur que x** (sur le même niveau)
- le **plus proche de la racine**

Mots qui veulent dire DFS

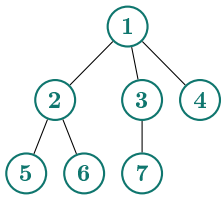
- **fils**, **père**, **sous-arbre**, **chemin**, **feuille**
- **hauteur**, taille, somme, moyenne
- profondeur **d'un nœud** ou **des feuilles**
- « parcours profondeur » imposé

Énoncé		Mot repère	Parcours
Chaque **niveau** a plus de nœuds que le précédent		niveau	**BFS**
Nombre max de nœuds sur un même **niveau**		niveau	**BFS**
Nœud de clé x le **plus proche de la racine**		plus proche	**BFS**
Clé de chaque nœud = somme des clés de ses **fils**		fils	**DFS**
Existe-t-il un **chemin** racine → feuille de somme s ?		chemin	**DFS**
Profondeur moyenne des **feuilles**	(piège)	feuilles	**DFS**

Pourquoi ? Ordre de visite sur le même arbre :



DFS : descend d'abord



BFS : niveau par niveau

- Le **BFS** donne tous les nœuds d'un niveau **à la suite** → on compare par niveau et on **s'arrête tôt**.
- Le **DFS** voit chaque nœud **avec ses fils** → idéal pour père/fils, sous-arbres, chemins.

Étape 2 – Les 4 questions avant d'écrire une ligne

1. **BFS ou DFS ?** → le mot repère ci-dessus.
2. **Que renvoie une feuille ?** → fixe le cas de base et la valeur de départ.
3. **Comment je combine les fils ?** → somme, `max`, « tous vrais » (CHAQUE), « au moins un » (IL EXISTE).
4. **Puis-je m'arrêter avant la fin ?** → si oui : condition dans le `while` ou `return` dans la boucle.

Les deux classes en un coup d'œil

	Tree	TreeAsBin
Est-ce une feuille ?	`T.nbchildren == 0`	`B.child is None`
Premier fils	`T.children[0]`	`C = B.child`
Fils suivant	`i += 1` puis `T.children[i]`	`C = C.sibling`
Parcourir tous les fils	`for C in T.children:`	`while C: ... C = C.sibling`
Nombre de fils	`T.nbchildren`	à compter (boucle)
« Arrivé au bout » (boucle while)	`i == T.nbchildren`	`C is None`
« Arrêté avant la fin »	`i < T.nbchildren`	`C is not None`

Comment lire les squelettes

Les mots **EN MAJUSCULES** (`DEPART`, `COMBINE`, `FIN`, `LOCAL_OK`, `TEST`) ne sont **pas** du Python : ce sont des **trous à remplir** selon l'exo. Chaque pattern est donné **à gauche en Tree** et **à droite en TreeAsBin**.

DFS 1 – Accumuler (taille, feuilles, hauteur)

On appelle la fonction sur **chaque fils**, on combine, et on fait `return` **après** la boucle.

Tree

```
def f(T):
    acc = DEPART        # ce que vaut une feuille
    for C in T.children:    # chaque fils C
        r = f(C)        # résultat du sous-arbre C
        acc = COMBINE(acc, r)  # ex : acc + r
                               #   ou max(acc, r)
    return FIN(acc)     # souvent juste acc
```

TreeAsBin

```
def f(B):
    acc = DEPART        # ce que vaut une feuille
    C = B.child         # premier fils
    while C:            # tant qu'il reste un fils
        r = f(C)        # résultat du sous-arbre C
        acc = COMBINE(acc, r)
        C = C.sibling   # fils suivant (obligatoire !)
    return FIN(acc)
```

Fonction	Départ	Combinaison	Fin	`if` feuille ?
taille	`s = 1`	`s += f(C)`	`s`	non
feuilles	`s = 0`	`s += f(C)`	`s`	oui : `return 1`
hauteur (feuille = 0)	`h = -1`	`h = max(h, f(C))`	`h + 1`	non
arité max	`k = m = 0`	`k += 1 ; m = max(m, f(C))`	`max(k, m)`	non
max des fils	`m = 1re clé`	`m = max(m, C.key)`	`m`	–

Le `if` feuille ne sert que si une feuille renvoie autre chose que la formule générale avec zéro fils. Pour un **max**, ne jamais partir de 0 (clés négatives) : partir du premier élément réel.

DFS 2 – Vérifier : « CHAQUE » ou « IL EXISTE » ?

Le mot de l'énoncé choisit la boucle

« **chaque** », « **tous** » ⇒ tous les fils doivent réussir ⇒ on s'arrête au **premier échec**.
« **existe-t-il** », « **au moins un** » ⇒ un seul fils suffit ⇒ on s'arrête au **premier succès**.
Rappel : la condition d'un `while` dit quand on **continue**, pas quand on s'arrête.

CHAQUE / TOUS

check_sum, check_max, heap_min, even_leaves, deep_enough

Tree

```
def check(T):
    if not LOCAL_OK(T):  # T respecte la règle ?
        return False
    i = 0
    while (i < T.nbchildren
           and check(T.children[i])):
        i += 1           # fils OK : au suivant
    return i == T.nbchildren  # au bout = tous OK
```

TreeAsBin

```
def check(B):
    if not LOCAL_OK(B):  # B respecte la règle ?
        return False
    C = B.child
    while C and check(C):  # tant que le fils RÉUSSIT
        C = C.sibling
    return C is None      # au bout = tous OK
```

IL EXISTE

path_sum, path_increasing, path_below, leaf_at_depth

Tree

```
def exists(T):
    if not LOCAL_OK(T):    # chemin mort ?
        return False
    if T.nbchildren == 0:  # feuille atteinte
        return True        # (ou le test final)
    i = 0
    while (i < T.nbchildren
           and not exists(T.children[i])):
        i += 1             # fils échoue : au suivant
    return i < T.nbchildren  # arrêté avant = trouvé
```

TreeAsBin

```
def exists(B):
    if not LOCAL_OK(B):    # chemin mort ?
        return False
    if B.child is None:    # feuille atteinte
        return True        # (ou le test final)
    C = B.child
    while C and not exists(C):  # tant que ÉCHEC
        C = C.sibling
    return C is not None   # arrêté sur un fils
```

Versions avec `return` dans la boucle (acceptées, souvent plus lisibles)

Tree

```
# CHAQUE                  # IL EXISTE
for C in T.children:      for C in T.children:
    if not check(C):          if exists(C):
        return False              return True
return True               return False
```

TreeAsBin

```
# CHAQUE                  # IL EXISTE
C = B.child               C = B.child
while C:                  while C:
    if not check(C):          if exists(C):
        return False              return True
    C = C.sibling             C = C.sibling
return True               return False
```

	CHAQUE	IL EXISTE
Condition du `while`	`f(fils)`	`not f(fils)`
`return` final Tree	`i == T.nbchildren`	`i < T.nbchildren`
`return` final TreeAsBin	`C is None`	`C is not None`
Cas feuille à écrire ?	seulement si la feuille a sa propre règle	**toujours** (sinon rien n'est trouvé)

Les deux sont **en miroir** : le `not` va **soit** dans le `while`, **soit** dans le `return`, jamais les deux, jamais aucun.

Où placer le test de la règle ? « Quels nœuds sont concernés ? »

Les feuilles seulement	→ dans le `if` feuille	ex. even_leaves : `return T.key % 2 == 0`
Tous les nœuds	→ au **début**, avant la boucle	ex. heap_min, path_below, deep_enough
Les nœuds internes	→ **après** la boucle	ex. check_sum : `... and s == T.key`

DFS 3 – Remonter un tuple (plusieurs infos, un seul parcours)

Tree

```
def __aux(T):
    """renvoie (ok, nb, somme) du
       sous-arbre T"""
    (nb, s) = (1, T.key)     # T tout seul
    ok = True
    i = 0
    while i < T.nbchildren and ok:
        (ok, n, sc) = __aux(T.children[i])
        ok = ok and TEST(n, sc, T)
                             # ex : sc / n <= T.key
        nb += n              # + ses noeuds
        s += sc              # + ses clés
        i += 1
    return (ok, nb, s)

def f(T):
    return __aux(T)[0]       # on garde ok
```

TreeAsBin

```
def __aux(B):
    """renvoie (ok, nb, somme) du
       sous-arbre B"""
    (nb, s) = (1, B.key)     # B tout seul
    ok = True
    C = B.child
    while C and ok:          # stop dès ok = False
        (ok, n, sc) = __aux(C)
        ok = ok and TEST(n, sc, B)
                             # ex : sc / n <= B.key
        nb += n              # + ses noeuds
        s += sc              # + ses clés
        C = C.sibling
    return (ok, nb, s)

def f(B):
    return __aux(B)[0]       # on garde ok
```

Deux parcours séparés (size puis somme) = **points perdus**. Docstring **obligatoire** : dire ce que vaut chaque case du tuple. Exemples TD : average_subtrees → `(ok, nb, somme)`, average_arity → `(nb internes, somme arités)`.

DFS 4 – Un paramètre qui descend

Le tuple **remonte** des fils vers le père ; ici l'info **descend** du père vers les fils.

Tree

```
def __aux(T, d):
    """d = profondeur de T (racine : 0)"""
    if T.nbchildren == 0:    # feuille :
        return d             # sa profondeur
    s = 0
    for C in T.children:
        s += __aux(C, d + 1)  # fils : 1 cran plus bas
    return s

def lce(T):
    return __aux(T, 0)       # on part de la racine
```

TreeAsBin

```
def __aux(B, d):
    """d = profondeur de B (racine : 0)"""
    if B.child is None:      # feuille :
        return d             # sa profondeur
    s = 0
    C = B.child
    while C:
        s += __aux(C, d + 1)  # fils : 1 cran plus bas
        C = C.sibling
    return s

def lce(B):
    return __aux(B, 0)
```

Exo	Paramètre	Vers le fils	Au départ (racine)
cheminement, deep_enough	profondeur `d`	`d + 1`	`0`
clés du niveau k	niveaux restants `k`	`k - 1`	`k` ; on prend la clé quand `k == 0`
chemin de somme s	reste `s`	`s - clé`	`s`
heap_min, path_increasing	clé du père `prev`	clé du nœud	faux père : `clé - 1`
liste à remplir	liste `L`	la même `L`	`[]` (puis `L.append(...)`)

Faut-il une fonction auxiliaire ? Écris la phrase d'abord (c'est la docstring exigée)

« Vrai si … dans le sous-arbre, **sachant que** … »

- Pas de « sachant que » ⇒ **pas d'auxiliaire** (hauteur, check_sum, path_sum : `s` est déjà dans l'énoncé).
- Il manque une info **d'au-dessus** (père, profondeur, reste…) ⇒ auxiliaire avec **un paramètre qui descend**.
- Il faut **plusieurs infos d'en dessous** (nombre ET somme) ⇒ auxiliaire qui **renvoie un tuple**.
- Jamais une fonction séparée pour « calculer la profondeur » : un nœud ne connaît pas son père, on la fait **descendre**.

BFS – Par niveaux (marqueur None ou 2 files)

La file mélange deux niveaux. On repère la **fin d'un niveau** : c'est **le seul endroit** où on compare et où on s'arrête.

Marqueur None

Tree

```
q = Queue()
q.enqueue(T)
q.enqueue(None)        # None = fin du niveau 0
cur = 0                # noeuds vus dans ce niveau
while not q.isempty():
    T = q.dequeue()
    if T is None:      # >>> FIN D'UN NIVEAU <<<
        ...            # ici : comparer, return...
        cur = 0
        if not q.isempty():  # sinon boucle infinie
            q.enqueue(None)
    else:
        cur += 1       # un noeud de plus
        for C in T.children:
            q.enqueue(C)     # niveau suivant
```

TreeAsBin

```
q = Queue()
q.enqueue(B)
q.enqueue(None)        # None = fin du niveau 0
cur = 0
while not q.isempty():
    B = q.dequeue()
    if B is None:      # >>> FIN D'UN NIVEAU <<<
        ...            # ici : comparer, return...
        cur = 0
        if not q.isempty():
            q.enqueue(None)
    else:
        cur += 1
        C = B.child
        while C:               # enfiler les fils
            q.enqueue(C)
            C = C.sibling
```

Deux files

Tree

```
q = Queue()
q.enqueue(T)
while not q.isempty():      # 1 tour = 1 niveau
    q_next = Queue()        # niveau suivant
    cur = 0
    while not q.isempty(): # 1 tour = 1 noeud
        T = q.dequeue()
        cur += 1
        for C in T.children:
            q_next.enqueue(C)
    ...                     # >>> FIN DU NIVEAU <<<
    q = q_next              # on passe au suivant
```

TreeAsBin

```
q = Queue()
q.enqueue(B)
while not q.isempty():      # 1 tour = 1 niveau
    q_next = Queue()
    cur = 0
    while not q.isempty(): # 1 tour = 1 noeud
        B = q.dequeue()
        cur += 1
        C = B.child
        while C:
            q_next.enqueue(C)
            C = C.sibling
    ...                     # >>> FIN DU NIVEAU <<<
    q = q_next
```

Exo	Gardé d'un niveau à l'autre	À la fin de chaque niveau
largeur	`w`	`w = max(w, cur)`
level_up	`prev` (départ 0)	`if cur <= prev: return False` puis `prev = cur`
max par niveau	liste `L`	`L.append(m)`
niveau de x	`lvl`	`lvl += 1` (et `return lvl` dès que x sort de la file)
niveaux croissants	`last`	`last = None`
levels (liste de listes)	`L + level = []` par niveau	`L.append(level)`

Variante échange : créer `q` et `qnxt` une seule fois en haut, puis à la fin de chaque niveau `q, qnxt = qnxt, q` (`q` est vide, elle resservira). Même complexité, les deux sont acceptées.

Piège levels : `level = []` ne monte **jamais** en haut : `L.append(level)` range la **même** liste, on obtiendrait `[[0, 4], [0, 4], [0, 4], [0, 4]]`.

Deux tests `q.isempty()` : l'intérieur = « reste-t-il des nœuds dans ce niveau ? », l'extérieur = « l'échange a-t-il apporté un niveau suivant ? ».

Attention : `Queue` n'a pas de `len` → on compte avec une variable (`cur`). On compare **à la fin du niveau**, jamais pendant qu'on enfile.

Parcours parallèle Tree / TreeAsBin

```
def symmetric(T, B):
    if T.key != B.key:      # clés différentes : perdu
        return False
    C = B.child             # B lu de gauche à droite
    i = T.nbchildren - 1    # T lu de droite à gauche
    while C and i >= 0 and \
          symmetric(T.children[i], C):
        i -= 1              # on avance des deux côtés
        C = C.sibling
    return C is None and i == -1
                            # les deux finis ensemble
```

- Un **indice** `i` avance dans `T.children`, un **pointeur** `C` avance sur les frères.
- Fin : **les deux épuisés en même temps**.
- `Tree` → `TreeAsBin` : parcourir les fils **de droite à gauche** et chaîner chaque copie devant la précédente (`C.sibling = first`).
- `TreeAsBin` → `Tree` : `T.children.append(copie(C))` pour chaque fils `C`.

Tes pièges (vus en séance)

Piège	Faux	Juste
Nœud ou liste de fils	`f(T.children)`	`for C in T.children: f(C)`
Tester une feuille	`T.children == 0`	`T.nbchildren == 0`
	`B.child == 0`	`B.child is None`
Nœud comparé à un nombre	`if C == 0`	`if C.key == 0`
`return` qui coupe la boucle	`for ...: return 1 + f(C)`	accumuler, `return` après
Mauvais départ	`s = 1` (feuilles), `m = 0` (max)	« le nœud compte-t-il ? », 1er élément
Frère / indice oublié	`while C:` sans `C.sibling`	`C = C.sibling / i += 1`
Appel récursif sur le même nœud	`f(B)` dans la boucle	`f(C)` (le fils)
Renommer sans changer l'appel	`def __aux3 ... __aux(C)`	même nom partout
CHAQUE / IL EXISTE inversés	`not` dans le `while` et le `return`	un seul `not`
IL EXISTE sans cas feuille	renvoie toujours `False`	`if feuille: return True`
Taille d'une file	`len(q)`	un compteur
Comparer trop tôt (BFS)	test dans le `for`	test à la fin du niveau

Checklist optimisation avant de rendre

- ☐ Arrêt dès que la réponse est connue
- ☐ Un seul parcours (tuple si besoin)
- ☐ Pas d'appel en double : `max(h, f(C))`
- ☐ Pas de liste construite pour rien
- ☐ Pas de `if` que le cas général gère
- ☐ Seulement `len range min max abs Queue`
- ☐ Fonctions auxiliaires documentées
- ☐ Dernière fonction = celle demandée